

Cross-Language Clone Generation and Evaluation Using GPTCloneBench

Chi Vu

University of Saskatchewan, Canada
c****6@usask.ca

Muhammad Khan

University of Saskatchewan, Canada
m****@usask.ca

Denys Schmitz

University of Saskatchewan, Canada
d****3@usask.ca

Ajmain Alam

University of Saskatchewan, Canada
n****9@usask.ca

ABSTRACT

As Large Language Models (LLMs) such as GPT and Llama become more prevalent in software development, their use in translating functionally similar code between programming languages has increased. This has contributed to the emergence of cross-language code reuse, where developers leverage LLMs to generate equivalent functions in different languages, leading to cross-language code clones. However, the quality, semantic consistency, and detectability of these LLM-generated clones remain underexplored. Traditional clone detection tools struggle with cross-language clones due to differences in syntax, paradigms, and other unique characteristics of each programming language.

This study evaluates the quality of LLM-generated cross-language clones and the effectiveness of embedding-based similarity analysis in detecting them. We constructed a dataset of over 6000 clone pairs across Java, Python, and C#, manually validating 10% of samples for semantic accuracy. Using cosine similarity and embedding models (StarCoder2-3B, CodeT5, and CodeT5+), we found that a 0.75 threshold balances precision and recall. Results show both GPT-3.5 and Llama3 generate accurate clones with over 88% overall accuracy.

Our contributions include a novel benchmark dataset consisting of cross-language code clones, a practical similarity-based evaluation pipeline to identify true clone pairs, and a comparative assessment to provide insight into the strengths and limitations of LLMs in generating and detecting cross-language clones. This work lays the foundation for further research into the development of more reliable and scalable methods for cross-language clone detection.

REPLICATION PACKAGE

The replication materials, including datasets and code, are included in the proceeding repository: <https://git.cs.usask.ca/das463/texcel>

CCS CONCEPTS

• **Software and its engineering** → *Software repository mining; Software maintenance and evolution; Practitioners' perspective; Software maintenance tools*; • **Computing methodologies** → *Machine learning*.

KEYWORDS

Cross-language code clones, Large Language Models, GPT, Llama, clone detection, embedding-based similarity analysis

ACM Reference Format:

Chi Vu, Denys Schmitz, Muhammad Khan, and Ajmain Alam. 2025. Cross-Language Clone Generation and Evaluation Using GPTCloneBench. In *Advanced Software Engineering (Winter 2025)*, April 2025, Saskatoon, Canada. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 INTRODUCTION

A **code clone** is a pair of code fragments that are identical or similar to each other, with major or minor modifications. Software developers often reuse code across different programming languages, resulting in code clones. These clones are frequently introduced via copying and pasting, porting logic across platforms, or translating functions manually. While code reuse can improve efficiency, it also raises challenges in maintaining correctness, consistency, and security across multilingual codebases. Studies estimate that 9–17% of code in a large software system are clones, which can lead to increased maintenance costs and fault proneness if not properly managed [20].

A particularly complex category of clones is **cross-language code clones**, which are functionally equivalent code written in different programming languages. These might differ in structure, but have the same functionality. Cross-language clones commonly appear in multi-platform development or when developers manually translate code between languages. They are difficult to detect due to differences in syntax, paradigms, and language idioms [16].

Rapid advancements in code generation have transformed software engineering. The success of models like GPT and Llama in generating human-like text raises questions about their potential to enhance code generation and improve developer efficiency. Code clone detection, a vital area of research, can also benefit from Large Language Models (LLMs). The increasing complexity of modern software systems necessitates innovative approaches to code analysis, making it essential to evaluate whether current clone detection methods can handle the nuances introduced by LLM-generated code.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CMPT 470, April, 2025, Saskatoon, Canada

© 2025 Association for Computing Machinery.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM...\$15.00

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

The recent rise of Large Language Models such as GPT-3.5 and Llama3 has accelerated the creation of cross-language clones. Roy et al. [13]. showcased GPT3 has large potential for cross language clone detection. We wanted to expand this to a model with similar performance such as Llama3:3b Developers now use LLMs to quickly translate or generate functionally equivalent code in different languages, either for porting features or exploring alternate implementations. Although this has improved productivity, it also introduces new challenges: the quality, semantic accuracy, and detectability of these clones are still not well understood.

Traditional tools often fail to identify such clones, as they rely on language-specific parsing and syntax-level comparisons that may miss deeper semantic relationships. Early approaches combine structural and nominal similarities to detect clones within a single language [11]. More recent methods use deep learning techniques [18] and code representations, such as graph neural networks (GNNs) [19] for detection across different languages. In GPTCloneBench [1], authors create a benchmark specifically for detecting cross-language clones. Other tools, such as C4 [16] and RUBHUS [9], also attempt to address this challenge. However, they face practical limitations, including slow processing speeds and unclear documentation, which make it difficult to adapt them to custom datasets. As a result, evaluating and detecting LLM-generated cross-language clones remains a critical gap in software engineering research.

To address this gap, we propose a new approach using embedding-based similarity analysis. This method extracts high-dimensional vector embeddings from pre-trained models and measures cosine similarity between code fragments to detect semantic clones. We evaluate the effectiveness of this method in detecting clones generated by GPT-3.5 and Llama3, using embedding models like StarCoder2-3B, CodeT5, and CodeT5+.

Our contributions are:

- A dataset of cross-language clones using GPT-3.5 and Llama3, spanning Java, Python and C# clone pairs.
- An embedding-based similarity analysis using cosine similarity with a suitable threshold for clone detection.
- An evaluation of three embedding models (StarCoder2-3B, CodeT5, CodeT5+) to compare detection effectiveness across different language pairs and LLM-generated datasets.
- An analysis of model-specific performance differences through statistical testing.

Our study aims to answer the following research questions:

RQ1: Can GPT-3.5 and Llama 3 generate semantically similar cross-language code clones given a benchmark dataset?

RQ2: How well do embedding-based similarity methods perform in identifying true cross-language clone pairs?

The findings of this research have the potential to improve several aspects of software engineering. Our work is expected to provide valuable insight into the behaviour and performance of clone detection tooling, which will enable the development of more accurate and robust detection systems. These improvements can lead to better identification of redundant or semantically similar code, which in turn supports more efficient code maintenance and refactoring efforts. Juergens et al. [6] showed that clones are harmful

```

Python:
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)
print(factorial(5))

Java:
public class Factorial {
    public static long factorial(int n) {
        if (n == 0 || n == 1) return 1;
        return n * factorial(n - 1);
    }

    public static void main(String[] args) {
        System.out.println(factorial(5));
    }
}

```

Figure 1: Cross-Language Clone Pair of Java and Python

to software systems, as inconsistent changes to clones occur quite frequently and cause a significant number of faults.

By building better clone detection tools, developers will be better equipped to assess clone quality and make informed decisions regarding code reuse and transformation. This could include ideas on what to monitor, how to validate clone integrity, and strategies for integrating automated clone generation into existing development workflows.

The rest of the paper is designed as follows: Section 2 discusses the related concepts for this work and Section 3 describes the methodology. In section 4 shows our results and section 5 discusses the findings. Section 6 showcases the related work with comparison and section 7 comments about the future direction of this work. Section 8 points out the threats of this research and section 9 concludes the paper.

2 BACKGROUND DETAILS

2.1 Definitions of Code Clones

Code clones can be categorized into four types:

- **Type-1 (Exact Clones):** Identical code snippets, except for whitespace or comments.
- **Type-2 (Renamed Clones):** Identical except for variable names, function names, or formatting changes.
- **Type-3 (Near-Miss Clones):** Similar in structure with small modifications such as added/removed statements.
- **Type-4 (Semantic Clones):** Functionally equivalent but structurally different implementations.

Cross-language clones typically fall into the **Type-4** category because they are functionally equivalent but structurally different. These clones arise when the same algorithm or logic is implemented in two or more different programming languages. This is common in multilingual systems, platform-specific adaptations (e.g., Java for Android and Swift for iOS), or situations where developers must maintain equivalent functionality across teams using different languages. In modern development pipelines, they may also emerge as a by-product of automated translation by LLMs or trans-compilers.

Figure 1 presents a code clone pair that implements a factorial function in both Java and Python. Although the syntax, structure, and idiomatic usage differ between the two languages, the logic remains consistent. This is what makes cross-language clones particularly difficult to detect. For instance, a recursive implementation in Java may be translated to an iterative form in Python, or a C-style loop might become a functional construct in a language like Scala. These variations break the assumptions of many existing tools, which expect structural similarity to imply semantic equivalence.

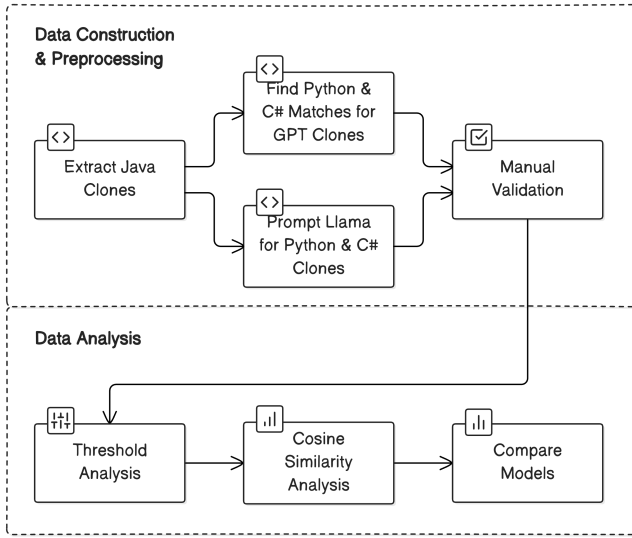


Figure 2: Flowchart of Code Clone Pair Generation and Validation

Furthermore, inconsistencies introduced during translation, especially by automated tools, can result in subtle bugs or behavioural deviations. Thus, detecting these clones is not just an academic exercise, but a practical necessity for ensuring correctness, consistency, and maintainability in multilingual codebases.

2.2 Embedding-Based Similarity

Embedding based similarity is a method of extracting the embeddings from a model for a given clone pair which works as follows:

- (1) Extract the embeddings from a specified model (StarCoder2-3B, CodeT5, CodeT5+)
- (2) Compute the distance by (cosine similarity, euclidean distance, or the dot product) for the generated embeddings
- (3) Classify a clone as a true pair if it meets or exceeds some threshold

The embedding models we used are specifically trained for code generation. An **embedding model** is a kind of ML model that transforms input data into a high dimensional vector representation. The distance metric used in our embedding-based similarity analysis was cosine similarity. **Cosign similarity** compares two vectors and outputs values in the range of $[-1,1]$. Vectors that score -1 are opposites and vectors that score 1 are identical to each other. Cosine similarity is given by the equation:

$$\cos(\theta) = \frac{A * B}{||A|| ||B||}$$

where A and B are vectors. **Z-Test** is a statistical test to determine whether two population means are different or to compare one mean to a hypothesized value when the variances are known and/or the sample size is large.

3 METHODOLOGY

This section describes the methodology of this research (Figure 2).

```

Convert the following Java code to {Python|C#}.
Respond only with valid {Python|C#} code. Try to
match the structure of the code as close as
possible (i.e., do not add extra whitespace if it
does not exist in the Java code but add it if it
does). Do not write an introduction or summary. It
is very important to only return a single function;
do not make any additional classes or functions:
{code_snippet}

```

Figure 3: Prompt sent to Llama to generate Python and C# clone from Java

3.1 Dataset & Data Characteristics

The dataset consists of two primary components: GPTClones sourced from the GPTCloneBench benchmark and LlamaClones generated specifically for this study using Llama. The GPTClones include Java source files with corresponding Python and C# variants, with at least 2,500 clones per target language for each Java file. For the LlamaClones, we prompted the Llama model with Java files from GPTCloneBench to produce three Python and three C# versions of each source file. The following criteria were applied during dataset selection:

- (1) All clones must maintain functional equivalence with the original Java code.
- (2) Only syntactically valid outputs are included.
- (3) The dataset is balanced between language pairs (Java $\rightarrow$ Python and Java $\rightarrow$ C#).

To standardize clone generation, we designed a structured prompt (Figure 3) instructing Llama to preserve functional equivalence. The model was asked to produce Python and C# clones that closely followed the structure of the original Java files, maintained validity, and preserved functional equivalence. We iterated on the prompt by testing different variations and refining it to obtain the correct output.

Once the prompt was finalized, we automated the generation, storage, and logging of clones using several Python scripts and processing libraries. The structure of the dataset is shown in Figure 10.

3.2 Data Pre-processing

To ensure the quality of the generated clones, 10% of the generated clone pairs (Java $\rightarrow$ Python and Java $\rightarrow$ C#) for each language were manually validated. We used Jeffrey Svajlenko’s clone validator tool, a Java GUI designed for side-by-side comparison of potential clone pairs¹. Each pair could be marked as true positive, false positive, or undecided. Clones were marked as true positives only if they preserved both functional logic and idiomatic conventions of the target language. The work was evenly divided among the validators, with each person reviewing one-third of each batch per language pair (Java $\rightarrow$ Python and Java $\rightarrow$ C#). In cases where a validator was unsure about the classification of a clone pair, the undecided clones were noted and discussed until a consensus was reached. If

¹<https://github.com/jeffsvajlenko/ValidateClones>

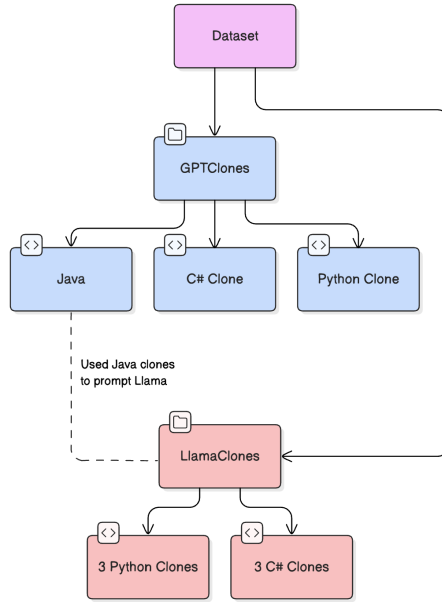


Figure 4: Dataset Structure

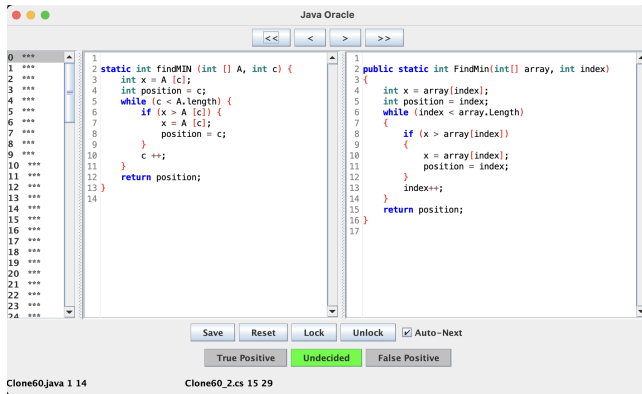


Figure 5: Clone Validator Java GUI

an agreement could not be made, those clones were discarded from the dataset.

Approximately 87% of the validated clones were identified as true positives, meaning they retained the original function's logic while adapting to the target language. The false positives were removed from the dataset for further analysis.

3.3 Threshold Determination

Due to the slow computation time and unclear documentation of current clone detection tools, embedding-based similarity was used to identify true clone pairs. While doing our embedding-based similarity analysis the cosine similarity distance metric was used. This distance metric was used for three main reasons compared to distance metrics such as euclidean distance or the dot product.

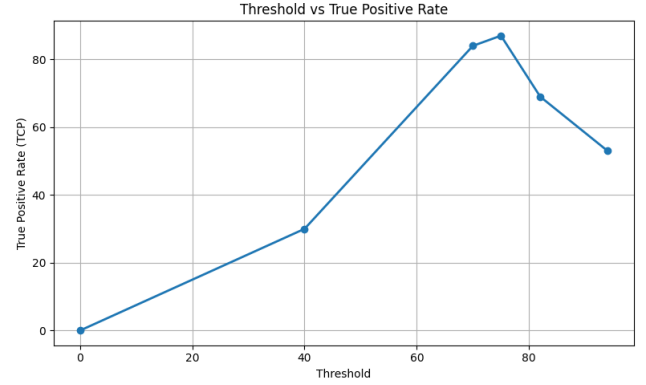


Figure 6: Threshold vs True Positive Rate

- (1) **Suitable for High dimensional Spaces:** with euclidean distance the distance between points usually increased with higher dimensionality. Due to the vector embeddings of transformer models is a high dimensional representation of the models input euclidean distance would be a poor choice for this task. Cosine similarity quantifies the angles between vectors and as such is less effected by high dimensional data.
- (2) **Robust to Varying Text Sizes:** Languages such as Java and C# have more boiler-plate language features that need to be implemented for a piece of code than a language such as Python. Due to this, the generated code clones have some variance in their length. Similarly to the above cosine similarity quantifies the angle between two vectors which is more robust for input vectors that have a variance in length.
- (3) **Precedence in Clone Detection:** Current state-of-the-art clone detection methods such as CLCSDSA [10] use cosine similarity to aid in clone determination. Our research investigates a purely semantic-based method without the use of Deep or ML models.

To determine this threshold, we performed manual validation in the same manner as above across a varying range of T's. We discovered that the optimal threshold was $T = 0.75$, as shown in Figure 6. In previous studies, there is no general consensus on the best threshold to use due to differences in embedding models. This determination is crucial to the performance of embedding-based similarity as demonstrated by Pinku et al. [12]. Generally across our research, we found that a cosine similarity threshold between 70% and 80% was usually a good determination for accuracy. Due to our task of identifying cross-language clones we found that a threshold of 80% was too strict. On our data set a T of 0.8 worked well on similar languages such as Java and C# however, was too strict for properly identifying Java->Python clones. A less strict T of 0.75 performed the best, as its semantic requirements allowed for Java->Python clones to be classified accurately despite the syntactical differences between the specified languages.

Tools & Technologies: We used a diverse set of tools and frameworks across different stages of our research pipeline. These include:

- **Data Construction & Extraction:** Python for data processing and visualization (NumPy, Pandas, Matplotlib, Seaborn,

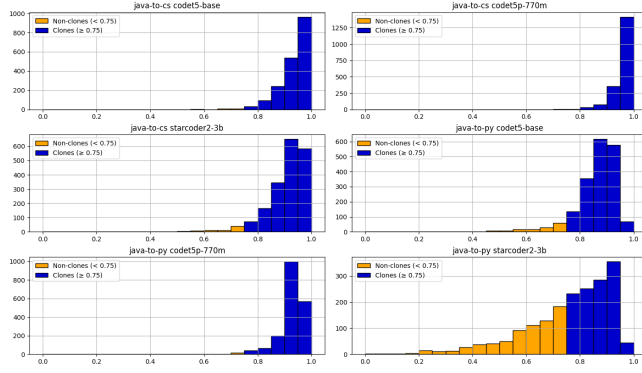


Figure 7: GPT Model Accuracy

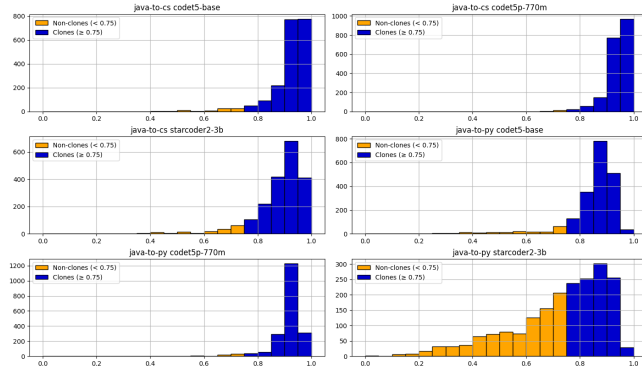


Figure 8: Llama Model Accuracy

etc.), GPTCloneBench benchmark, GPT and Llama models for clone generation.

- **Embedding Generation:** StarCoder2-3B, CodeT5, and CodeT5+ for code embeddings.
- **Similarity Analysis:** Cosine similarity for comparing embeddings and determining true clone pairs.
- **Evaluation Metrics:** Manual checks, threshold analysis, cosine similarity, and statistical significance testing

4 RESULTS

Model	Java → C# Accuracy	Java → Python Accuracy
StarCoder2-3B	95.97%	61.97%
CodeT5-base	94.06%	88.22%
CodeT5+	94.96%	94.26%

Table 1: Model accuracy for Java to C# and Java to Python translations using GPT-3.5.

Overview of Findings: RQ1: Can GPT-3.5 and Llama 3 generate semantically similar cross-language code clones given a benchmark dataset?

The overall accuracies for GPT-3.5 and Llama3 were 88.24% and 88.26%, respectively, as shown in Table 3. After performing a Z test

- GPT accuracy 1: 88.24% with $n_1 = 1952$

Model	Java → C# Accuracy	Java → Python Accuracy
StarCoder2-3B	92.15%	54.18%
CodeT5-base	96.07%	91.14%
CodeT5+	99.09%	96.98%

Table 2: Model accuracy for Java to C# and Java to Python translations using Llama3.

Model Family	Overall Accuracy
GPT	88.24%
Llama	88.84%

Table 3: Overall average accuracy across all evaluated tasks for GPT and Llama models.

- Llama accuracy 2: 88.84% with $n_2 = 5736$

Let $x_1 = 1722$ and $x_2 = 5096$ be the number of correct predictions.

$$p = \frac{x_1 + x_2}{n_1 + n_2} = \frac{1722 + 5096}{1952 + 5736} = \frac{6818}{7688} \approx 0.8868$$

$$SE = \sqrt{p(1-p) \left(\frac{1}{n_1} + \frac{1}{n_2} \right)} \approx 0.00621$$

$$z = \frac{p_1 - p_2}{SE} = \frac{0.8824 - 0.8884}{0.00621} \approx -0.9662$$

$$p\text{-value} \approx 0.3336$$

There are no statistically significant differences between the two accuracies.

RQ2 How well do embedding-based similarity methods perform in identifying true cross-language clone pairs?

- (1) **Model-Specific Performance Variance:** StarCoder2-3B showed reduced effectiveness, particularly for Java→Python clones in both datasets, with a notable performance drop (66.74% → 54.18%) on LlamaClones. CodeT5+ (770M) exhibited the most robust performance, achieving 94.26% accuracy for Java to Python in GPTClones (improving from initial 88%) and exceptional 99.09% accuracy for Java→C# in LlamaClones.
 - (2) **Language Pair Dependencies** Detection accuracy varied significantly between language pairs, with Java → C# consistently outperforming Java → Python in GPTClones and exceptional at 99.09% accuracy for Java → C# in LlamaClones. The performance gap between language pairs was most pronounced in StarCoder2-3B's results, suggesting varying capability in handling different syntactic transformations demonstrated by Figures 7 and 8.
 - (3) **Dataset-Specific Patterns** Llama-generated clones were generally easier to detect for Java → C#. GPTClones showed more consistent performance across language pairs. The largest performance gap between datasets occurred with StarCoder2-3B on Java → Python (61.97% vs 54.18%).
- Limitations and Uncertainties:**

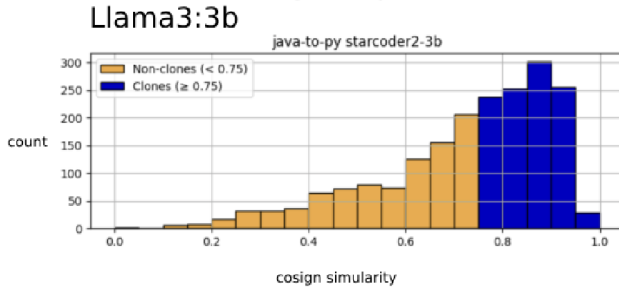


Figure 9: Cosine Similarity vs. Count for Embeddings for Llama3:3b

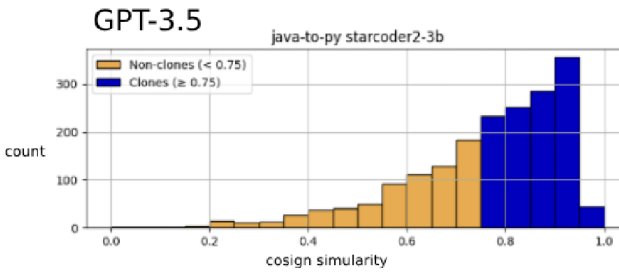


Figure 10: Cosine Similarity vs. Count for Embeddings for GPT-3.5

5 DISCUSSION

5.1 Key Findings

Our evaluation shows that both GPT-3.5 and Llama generate semantically similar cross-language clones with similar overall accuracies (88.24% for GPT-3.5 and 88.84% for Llama). The embedding-based similarity approach, using a cosine similarity threshold of 0.75, was effective in identifying true clones. Although the CodeT5+ model consistently performed better in both Java-to-C# and Java-to-Python tasks, there were no statistically significant differences between GPT-3.5 and Llama overall.

5.2 Interpretation of Results

RQ1: The lack of statistical significance between GPT-3.5 and Llama-3 indicates that both models are capable of generating functionally accurate cross-language clones. This was an expected outcome as we specifically evaluated similar models for the task of cross-language clone generation. Henydrcks et al. [5] introduced the MMLU benchmark which determines the general performance of LLMs across a varying set of domain-specific tasks for example mathematical reasoning or code generation. Using this benchmark GPT-3.5 and Llama3:3b scored 67.5 and 63.4 respectively. While GPT-3.5 performed better than the Llama3:3b model, these scores are similar. Generally, to see significant performance increase with this benchmark, the MMLU scores need to have a more significant gap in scores than what was observed with the GPT-3.5 Llama3:3b models.

RQ2: How well do embedding-based similarity methods perform in identifying true cross-language clone pairs? We demonstrated a purely vector embedding-based approach is beneficial for two key reasons:

- (1) **Industry is Outpacing:** Vector embeddings, which are derived from training machine learning models, offer a compact and semantically rich representation of source code. The rapid pace of advancement in LLM development in the industry frequently surpasses that of academic research. Within starting even this research 3 months ago more modern and capable GPT, Llama, and DeepSeek models were researched and brought to industry.
- (2) **Advanced ML Overhead:** The manual and computational overhead associated with deep learning techniques for detecting cross-language code clones using the state of the art techniques is non-trivial. Unclear documentation and machine specific dependencies caused many errors when trying to use tools like C4 [16] or RUBHUS [9] and were computationally expensive. This underscores the need for efficient, embedding-based methods in cross-language clone detection.
- (3) **Simplicity:** Once vector embeddings are generated, they preserve the semantic content of the data, eliminating the need for additional post-processing techniques. Moreover, the embeddings can be stored and reused, avoiding the need for repeated computation. DeepSeek-AI et al. [2] have demonstrated that LLMs can achieve strong performance by leveraging pre-trained models, reinforcing the effectiveness of this approach.

The computational overhead associated with deep learning techniques for detecting cross-language code clones with state-of-the-art techniques is non-trivial. This underscores the need for efficient, embedding-based methods in cross-language clone detection. A further advantage of this methodology is its simplicity: once vector embeddings are generated, they preserve the semantic content of the data, eliminating the need for additional post-processing techniques. Moreover, the embeddings can be stored and reused, avoiding the need for repeated computation. DeepSeek-AI et al. [2] have demonstrated that LLMs can achieve strong performance by leveraging pretrained models, reinforcing the effectiveness of this approach.

Limitations and Areas for Improvement There are several limitations to our study. First, the manual validation process, while thorough, may involve some subjectivity and could be improved with more automated or consensus-based methods. Second, we set the cosine similarity threshold ($T = 0.75$) based on our experiments. In the future, it might be useful to try adaptive thresholds or other similarity measures to further boost detection accuracy. Third, the current dataset, although balanced across language pairs, might not fully represent the variety found in real-world code. Future studies should consider expanding the dataset to include a wider range of programming styles and paradigms. Lastly, our analysis only focused on GPT-3.5 and Llama but examining additional language models may provide further insight into model-specific strengths and weaknesses. Overall, the results contribute to our understanding of cross-language clone generation and detection

and suggest that further refinement of both evaluation methods and clone detection tools is warranted.

6 RELATED WORK

Existing studies predominantly examine individual language models without directly comparing their performance in clone generation and detection. Our research addresses this limitation by constructing an expanded dataset that enables a direct comparison between the Llama and GPT models. By generating multiple outputs per input using the Llama model, we aim to analyze how different models interpret and transform code, offering a deeper understanding of their comparative effectiveness in various software engineering contexts.

6.1 Syntactic and Lexical-Based Approaches

Early approaches to clone detection primarily relied on syntactic and lexical analyses. For instance, the CLCDSA model introduced by Kawser et al. [10] combines syntactic features with a manual API mapping between languages such as Java and Python. Although this method can capture clones with significant lexical overlap, the process of manual API mapping is both time-consuming and error-prone. Given the rapid evolution of programming languages, such manual efforts are impractical for handling all emergent languages. Consequently, these methods are limited in their ability to generalize across diverse codebases and often struggle with detecting clones that exhibit deep semantic equivalence despite superficial syntactic differences.

6.2 AST and Parse-Tree-Based Approaches

To address the limitations of purely lexical methods, researchers have explored techniques that leverage Abstract Syntax Trees (ASTs) and other parse-tree representations. These approaches extract structural information from code to detect clones that share control-flow characteristics. In this domain, tools like RUBHUS [9] adopt a semi-supervised learning model that uses enriched AST representations—including control and data flow edges—to capture both syntactic and limited semantic information. While such methods can better identify clones that differ in surface-level syntax, they heavily rely on high-quality, language-specific parsers and on labeled data, which can be costly and time-consuming to obtain.

6.3 Graph-Based and Intermediate-Representation Approaches

Graph-based methods represent another significant direction in clone detection research. Tools leveraging graph neural networks (GNNs) or other graph representations, as discussed in [17] and [19], model code as graphs that capture intricate structural relationships, such as function calls and control-flow paths. These approaches have the potential to uncover deep semantic similarities that traditional methods might overlook. However, the construction and alignment of such graphs can be computationally intensive and complex, particularly when the code originates from different languages with distinct structural characteristics.

6.4 Deep-Learning and Embedding-Based Approaches

More recently, deep-learning techniques have been applied to clone detection by mapping code snippets into a shared embedding space. Methods such as C4 [16] and ZC³ [7] use contrastive learning and pre-trained multilingual models to bridge the gap between syntactic differences and capture semantic equivalence. Additional approaches, including XCode, TCCCD, and FSD-CLCD [3, 8, 19], employ transformer-based architectures and other neural models to generate robust code representations. While these embedding-based methods significantly improve the detection of semantically similar clones, they often require extensive training data and can suffer from reduced accuracy when confronted with out-of-domain code or when subtle semantic variations exist.

6.5 Existing Clone Benchmarks

Several existing benchmarks have been developed for code clone detection, but they exhibit limitations in their applicability for cross-language scenarios. BigCloneBench introduced by Svajlenko et al. [14] is one of the most widely recognized benchmarks for clone detection. However, it was later found to be particularly suited for detecting **Type 3** clones rather than semantic clones, making it less effective for a broader range of clone types [15]. Another benchmark, SemanticCloneBench introduced by Al-Omari et al. [4], aimed to address semantic clone detection. Despite its intent, this benchmark was found lacking in several aspects, including an insufficient number of code pairs to effectively train machine learning models. It also suffers from syntactic issues and contains a mixture of C++ paradigms in its C language clones [1]. These shortcomings highlight the need for more robust, comprehensive, and standardized datasets that can support both syntactic and semantic clone detection tasks across various programming languages. The dataset we generated using LLama3:3b generated 2868 **Type 4** clone pairs for the Java -> C# and Java -> Python languages.

This study contributes to the field by constructing a novel dataset comprising generated code clones from two distinct language models, which can serve as a resource for future evaluations of clone detection tools. Additionally, as previously mentioned, we will generate multiple outputs per input using the Llama model to examine the subtleties of its code interpretation processes. This analysis will provide valuable insights into the behaviour of LLMs in software engineering applications.

Our research aims to introduce the following key advancements:

- A more comprehensive and diverse dataset for evaluating code clone detection methodologies, integrating outputs from multiple language models.
- An increased sample size through the generation of multiple outputs per input, facilitating more robust statistical analysis.
- A refined understanding of model-specific characteristics in code generation, which may inform the development of more effective automated software engineering tools.

7 FUTURE WORK

There are three areas in which we would reasonably expand the work

- (1) **Expanded Model Comparison:** our current work only establishes a benchmark for how well the GPT-3.5 and Llama3:3b models generate cross-language clones. Expanding this work to other models such as Gemini or Grok would aid in the exploration of the semantic space cross-language clones represent.
- (2) **Expand Embedding Models:** The current work provides insights into how transformer models trained with coding specific data perform at this task. Comparison against other encoder models not specialized on code clones would inform if the use of code-specific transformers is necessary in this style of classification.
- (3) **Source Code Augmentation:** Investigate data augmentation methods such as variable/function name obfuscation or removing language primitives that distract from the syntactic and semantic representation of a code clone. Further exploration of this topic helps to generate embeddings that have a stronger semantic connection.

8 THREATS TO VALIDITY

This section discusses potential threats that may affect the credibility, generalizability, or interpretation of our results and suggests measures to mitigate them.

- **Internal Validity:** One threat to internal validity is the manual validation process used to classify clone pairs as human judgment may introduce subjectivity. To mitigate this, we divided the work evenly among multiple group members and discussed any undecided cases until consensus was reached. Additionally, the selection of the cosine similarity threshold ($T = 0.75$) may have affected the detection results.
- **External Validity:** Our study relies on a dataset derived from a specific benchmark (GPTCloneBench) and a controlled set of prompts for generating clones. This could limit the ability to generalize our findings to other codebases or programming paradigms not represented in our dataset. To improve external validity, future research should consider using a wider variety of source files and expanding the dataset to cover additional programming languages and coding styles found in real-world applications.
- **Construct Validity:** While we used overall accuracy and cosine similarity as key metrics, these measures may not fully reflect the functional or semantic equivalence of code clones.

9 CONCLUSION

Cross-language clone detection remains a challenging problem in software engineering, especially with the increasing use of LLMs for code generation. This study offers a comprehensive evaluation of cross-language code clone generation and detection using Large Language Models, specifically GPT-3.5 and Llama3. Our findings show that both models are capable of producing semantically accurate clones across different languages, with overall accuracies exceeding 88% and no statistically significance between them. This suggests modern LLMs are reliable tools for generating functionally similar cross-language code clones. Moreover, the effectiveness

of embedding-based similarity analysis demonstrates that lightweight, scalable, and semantically-informed clone detection methods can be viable alternatives to traditional, computationally intensive approaches. Our results show that embedding models, such as CodeT5+, consistently outperform others across language pairs, suggesting their utility in practical development and maintenance environments. The implications of this work are significant. First, it highlights the importance of creating robust benchmarks like GPTCloneBench to measure and compare cross-language clones. Second, it supports the industry's shift towards using LLMs for code analysis. Lastly, it emphasizes the need for continued refinement of detection tools that can handle multilingual and diverse codebases. Looking ahead, researchers can expand this work with further exploration of other model comparisons, alternative analysis methods, and more diverse datasets. These directions help us move toward smarter, AI-assisted tools for writing and maintaining code.

REFERENCES

- [1] Ajmain I Alam, Palash R Roy, Farouq Al-Omari, Chanchal K Roy, Banani Roy, and Kevin A Schneider. 2023. GPTCloneBench: A comprehensive benchmark of semantic clones and cross-language clones using GPT-3 model and SemanticCloneBench. In *2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 1–13.
- [2] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, and others. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948 [cs.CL]. <https://arxiv.org/abs/2501.12948>
- [3] Yong Fang, Fangzheng Zhou, Yijia Xu, and Zhonglin Liu. 2023. TCCCD: Triplet-Based Cross-Language Code Clone Detection. *Applied Sciences* 13, 21 (2023), 12084.
- [4] Chanchal K Roy Farouq Al-Omari and Tonghao Chen. 2020. Semanticclonebench: A semantic code clone benchmark using crowdsourced knowledge. In *In 2020 IEEE 14th International Workshop on Software Clones (IWSC)*, 57–66.
- [5] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring Massive Multitask Language Understanding. arXiv:2009.03300 [cs.CY]. <https://arxiv.org/abs/2009.03300>
- [6] Elmar Juergens, Florian Deissenboeck, Benjamin Hummel, and Stefan Wagner. 2009. Do code clones matter?. In *2009 IEEE 31st International Conference on Software Engineering*. IEEE, 485–495.
- [7] Jia Li, Chongyang Tao, Zhi Jin, Fang Liu, and Ge Li. 2023. ZC 3: Zero-shot cross-language code clone detection. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 875–887.
- [8] Zehao Lin, Guodun Li, Jingfeng Zhang, Yue Deng, Xiangji Zeng, Yin Zhang, and Yao Wan. 2022. Xcode: Towards cross-language code representation with large-scale pre-training. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 31, 3 (2022), 1–44.
- [9] Nikita Mehrotra, Akash Sharma, Anmol Jindal, and Rahul Purandare. 2023. Improving Cross-Language Code Clone Detection via Code Representation Learning and Graph Neural Networks. *IEEE Transactions on Software Engineering* 49, 11 (2023), 4846–4868. <https://doi.org/10.1109/TSE.2023.3311796>
- [10] Kawser Wazed Nafi, Tonny Shekha Kar, Banani Roy, Chanchal K Roy, and Kevin A. Schneider. 2019. CLCDSA: Cross Language Code Clone Detection using Syntactical Features and API Documentation. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 1026–1037.
- [11] Lawton Nichols, Mehmet Emre, and Ben Hardekopf. 2019. Structural and nominal cross-language clone detection. In *Fundamental Approaches to Software Engineering: 22nd International Conference, FASE 2019, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2019, Prague, Czech Republic, April 6–11, 2019, Proceedings* 22. Springer, 247–263.
- [12] Subroto Nag Pinku, Debajyoti Mondal, and Chanchal K Roy. 2023. Pathways to leverage transcompiler based data augmentation for cross-language clone detection. In *2023 IEEE/ACM 31st International Conference on Program Comprehension (ICPC)*. IEEE, 169–180.
- [13] Palash R Roy, Ajmain I Alam, Farouq Al-Omari, Banani Roy, Chanchal K Roy, and Kevin A Schneider. 2023. Unveiling the potential of large language models in generating semantic and cross-language clones. In *2023 IEEE 17th International Workshop on Software Clones (IWSC)*. IEEE, 22–28.
- [14] Jeffrey Svajlenko and Chanchal K Roy. 2021. Bigclonebench. Code Clone Analysis: Research, Tools, and Practices. 93–105.
- [15] Jeffrey Svajlenko and Chanchal K Roy. 2022. Bigclonebench: A retrospective and roadmap. In *2022 IEEE 16th International Workshop on Software Clones (IWSC)*. IEEE, 8–9.

- [16] Chenning Tao, Qi Zhan, Xing Hu, and Xin Xia. 2022. C4: Contrastive cross-language code clone detection. In *Proceedings of the 30th IEEE/ACM international conference on program comprehension*. 413–424.
- [17] Tijana Vislavski, Gordana Rakić, Nicolás Cardozo, and Zoran Budimac. 2018. LICCA: A tool for cross-language clone detection. In *2018 IEEE 25th international conference on software analysis, evolution and reengineering (SANER)*. IEEE, 512–516.
- [18] Mohammad A Yahya and Dae-Kyoo Kim. 2023. CLCD-I: cross-language clone detection by using deep learning with infercode. *Computers* 12, 1 (2023), 12.
- [19] Yuguo Zhang, Jia Yang, and Ou Ruan. 2024. Cross-language source code clone detection based on graph neural network. In *Proceedings of the 2024 3rd International Conference on Cryptography, Network Security and Communication Technology*. 189–194.
- [20] Minhaz F Zibran, Ripon K Saha, Muhammad Asaduzzaman, and Chanchal K Roy. 2011. Analyzing and forecasting near-miss clones in evolving software: An empirical study. In *2011 16th IEEE international conference on engineering of complex computer systems*. IEEE, 295–304.